RecruitBot

AI-Powered Recruitment Chatbot

1. Executive Summary

RecruitBot is an AI-powered recruitment assistant that enables hiring teams to query and analyse candidate data using natural language. Instead of manually filtering spreadsheets, recruiters can simply ask questions like "Who is the best candidate for the ML role?" or "Show me all candidates with Python skills" and receive instant, intelligent answers.

The system is built on a modern no-code/low-code stack, combining n8n for workflow automation, Google Sheets as a live candidate database, and OpenAI's GPT-4o as the AI brain, all accessible through a clean, browser-based chat interface.

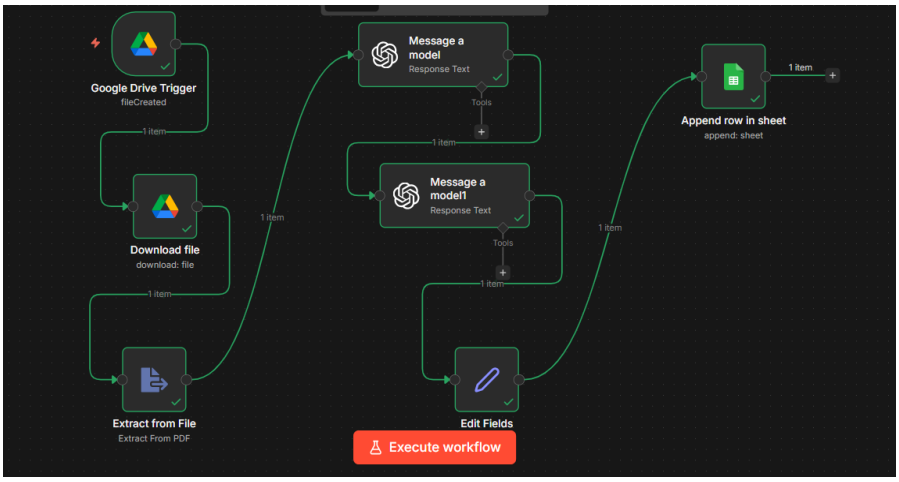
3. System Architecture

RecruitBot is composed of two automated workflows and a frontend chat interface:

Workflow 1: Resume Scoring

Automatically evaluates incoming candidate resumes and writes scored results to Google Sheets.

- Receives candidate data via webhook trigger
- Scores each candidate using GPT-4o based on skills, experience, and fit
- Writes name, email, score, category, skills, and reason to Google Sheets



1. Google Drive Trigger

- **Role:** Watches for new resumes being uploaded
- **Input:** Someone drops a resume PDF into a folder named: Resumes Incoming
- **Output:** File metadata of the newly uploaded resume

2. Download File

- **Role:** Downloads the actual resume PDF
- **Input:** File ID from the trigger
- **Output:** The raw binary PDF of the resume

3. Extract from File

- **Role:** Pulls all text out of the resume PDF
- **Input:** Binary PDF
- **Output:** Raw resume text (name, experience, skills, education etc. all as one text blob)

4a. Message a Model *(top)*

- **Role:** Likely the **primary resume parser**, extracts structured fields
- **Input:** Raw resume text
- **Output:** Structured data like name, email, phone, skills, years of experience, current role, education

4b. Message a Model1 *(bottom)*

- **Role:** The **scoring / evaluation model**, scores the candidate's resume, puts it in either of the three categories (approved, on hold and rejected) with a proper reason.
- **Input:** Same raw resume text
- **Output:** A score, resume category with reason

5. Edit Fields

- **Role:** Merges outputs from both AI models into exact Google Sheet column names
- **Input:** Structured data + score from both models
- **Output:** Row-ready object matching your sheet's columns

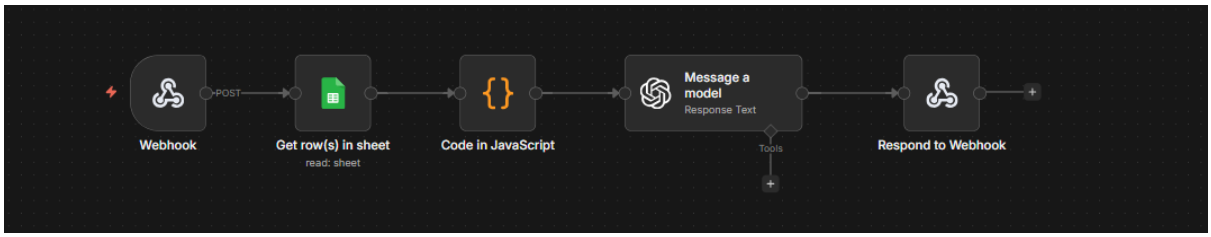
6. Append Row in Sheet

- **Role:** Logs the parsed candidate into your tracker sheet
- **Input:** Mapped fields
- **Output:** A new row with all candidate details added to the sheet

Workflow 2: RecruitBot Chatbot

Handles natural language queries from the chat UI and returns AI-generated answers.

- Webhook node receives POST request from the frontend
- Google Sheets node reads all candidate rows in real time
- Code node bundles the user query and candidate data together
- OpenAI GPT-4o generates a contextual, accurate response
- Respond to Webhook node returns the answer as JSON to the frontend



1. Webhook *(POST)*

- **Role:** Entry point that listens for incoming queries
- **Input:** A POST request, from chatbot interface, containing a **question**.
- **Output:** The incoming query/message payload
- **Important:** This is what makes the workflow **on-demand** i.e. it only runs when someone asks something

2. Get Row(s) in Sheet *(read: sheet)*

- **Role:** Fetches all candidate data from your Google Sheet
- **Input:** Triggered by the webhook (no filter, reads all rows)
- **Output:** All parsed resume rows, every candidate's name, skills, experience, score etc.
- **Important:** This is how the AI gets **context**, it reads the entire candidate database before answering

3. Code in JavaScript

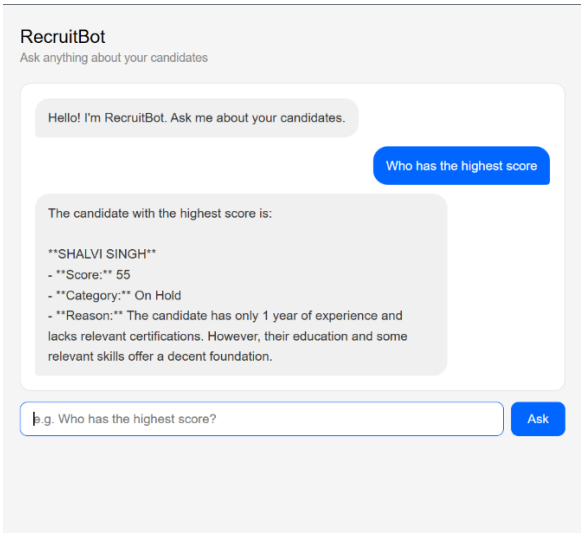
- **Role:** Formats and structures the sheet data into a clean prompt-ready format
- **Input:** Raw rows from the Google Sheet
- **Output:** A neatly formatted string/object that the AI model can consume
- **Important:** Converts the sheet rows JSON so the AI can reason over it properly

4. Message a Model *(Response Text)*

- **Role:** The **AI brain**, answers the user's question using the candidate data
- **Input:** The formatted candidate data + the original question from the webhook
- **Output:** A natural language answer like *"Shalvi has 5 years of Python experience and scored the highest among all candidates"*
- **Important:** This is essentially a **RAG-lite pattern**, stuffing your sheet data into the context window and asking questions over it

5. Respond to Webhook

- **Role:** Sends the AI's answer back to whoever made the request
- **Input:** AI response text
- **Output:** HTTP response returned to the caller (your frontend/chat UI/Postman)
- **Important:** Closes the request loop, without this, the caller would time out waiting

Frontend Chat Interface

A lightweight HTML/CSS/JavaScript chat UI served via VS Code Live Server. Sends POST requests to the n8n webhook and displays AI responses in a clean bubble-style chat layout.